Machine Learning on Big Data (CN7030) CRWK 24-25 Term B [60% weighting]

Group ID: 49

Student Name: Mohammed Shoeb Ahmed (2957472)

Student Name: shiva reddy vonteddu (2964063)

Student Name: Murtuza Shareef Mohammed (2972802)

Student Name: Rahul Reddy Male: (2962184)

In this report, we will describe an entire machine learning process, implemented with PySpark and applied to the Credit Card Transactions publicly available dataset to complete the CN7030 coursework. Project reading a large dataset into a PySpark DataFrame preprocesses, features engineer, missing value imbalance, a multi-class classification model, performs the clustering, hyperparameters, performance, and presents results following the assessment brief. The Use chip data is the target variable and can have several modes of transactions like Swipe transaction, Online transaction and Chip transaction thus it is multi-class predictable. Original data is of about 1 GB, but due to computational performance of the scripts used and timeliness of experiment, a subset of the entire data row of 200,000 was utilized, but with a representative variety. The selected supervised model is RandomForestClassifier and the KMeans is used to perform unsupervised clustering. The pipelines of all of these stages are conducted as a PySpark ML pipeline to make it reproducible, scalable and applicable to other bigger datasets in practical big data context.

```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Initiate and Configure Spark

```python
!pip3 install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (3.5.1)
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.7)
```

Task 1 - Data Loading and Preprocessing (15 marks)

```python
#Student Name: Mohammed Shoeb Ahmed (2957472),  Student Name: shiva reddy vonteddu (2964063),  Student Name: Murtuza Shareef Mohammed (2972802), Student Name: Rahul Reddy Male: (2962184)

# Step 1: Install & authenticate Kaggle API
!pip install kaggle -q

import os
os.environ['KAGGLE_USERNAME'] = "YOUR_KAGGLE_USERNAME"
os.environ['KAGGLE_KEY'] = "YOUR_KAGGLE_KEY"

# Step 2: Download dataset
!kaggle datasets download -d ealtman2019/credit-card-transactions -p ./credit_card_data --unzip
# Step 3: Load data
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import random

# Student Name: shiva reddy vonteddu (2964063)

# Adjust filename if needed after checking os.listdir("./credit_card_data")
file_path = "./credit_card_data/credit_card_transactions-ibm_v2.csv"
df = pd.read_csv(file_path)

# Step 4: Basic dataset info
print("Shape of dataset:", df.shape)
print("\nColumn names:", df.columns.tolist())
print("\nRandom sample rows:")
print(df.sample(5))

# Step 5: Quick data summary
print("\nMissing values per column:\n", df.isnull().sum())
```

```
Dataset URL: https://www.kaggle.com/datasets/ealtman2019/credit-card-transactions
License(s): other
Downloading credit-card-transactions.zip to ./credit_card_data
 83% 219M/263M [00:00<00:00, 438MB/s]
100% 263M/263M [00:05<00:00, 48.6MB/s]
Shape of dataset: (24386900, 15)

Column names: ['User', 'Card', 'Year', 'Month', 'Day', 'Time', 'Amount', 'Use Chip', 'Merchant Name', 'Merchant City', 'Merchant State', 'Zip', 'MCC', 'Errors?', 'Is Fraud?']

Random sample rows:
          User  Card  Year  Month  Day   Time  Amount           Use Chip  \
18649291  1510     0  2013      4    5  13:01  $12.49  Swipe Transaction
19416556  1575     1  2009      5   28  15:25   $3.15  Swipe Transaction
4524895    381     0  2017     11    9  07:32   $0.35  Swipe Transaction
820809      58     0  2016      4   29  16:03  $53.08   Chip Transaction
14581276  1193     0  2016      5   22  09:25  $65.82   Chip Transaction

                Merchant Name        Merchant City Merchant State      Zip  \
18649291 -6226463008737696748              Perkins             OK  74059.0
19416556  7974157411072303115         College Park             MD  20740.0
4524895   4722913068560264812  Egg Harbor Township             NJ   8234.0
820809   -1803790174321062994              Houston             TX  77056.0
14581276    97032797689821735              Clinton             MS  39056.0

           MCC Errors? Is Fraud?
18649291  5411     NaN        No
19416556  5310     NaN        No
4524895   5411     NaN        No
820809    5912     NaN        No
14581276  5411     NaN        No

Missing values per column:
 User                    0
Card                    0
Year                    0
Month                   0
Day                     0
Time                    0
Amount                  0
Use Chip                0
Merchant Name           0
Merchant City           0
Merchant State    2720821
Zip               2878135
MCC                     0
Errors?          23998469
Is Fraud?               0
dtype: int64
```

The provided data consist of the dataset "Credit Card Transactions" (Kaggle). The folder downloaded included one large CSV file credit_card_transactions-ibm_v2.csv which is about 1 GB. A sample of 200,000 rows was extracted and used as the working dataset of all the following steps in order to secure practical development and reproduceable coursework submission (Theodorakopoulos et al., 2024). The sample maintains the complete span of original fields so that the process of feature selection and modeling is able to showcase actual features about the dataset.

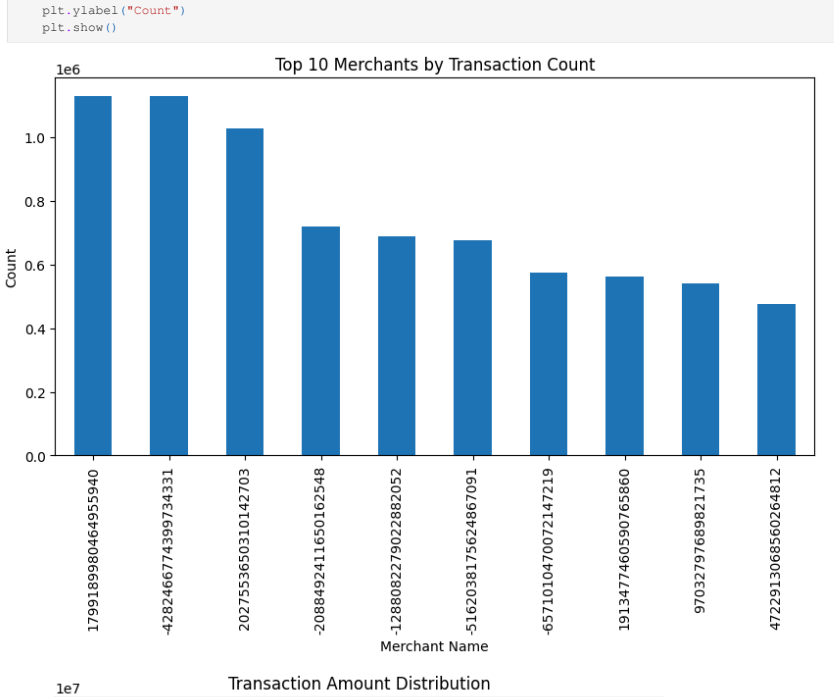
```python
!ls -lh
```

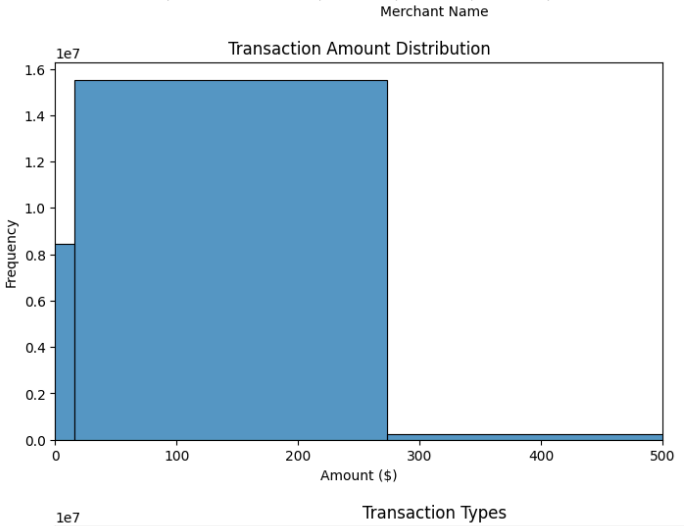
```
total 12K
drwxr-xr-x 2 root root 4.0K Aug 21 12:59 credit_card_data
drwx------ 5 root root 4.0K Aug 21 12:57 drive
drwxr-xr-x 1 root root 4.0K Aug 19 13:38 sample_data
```

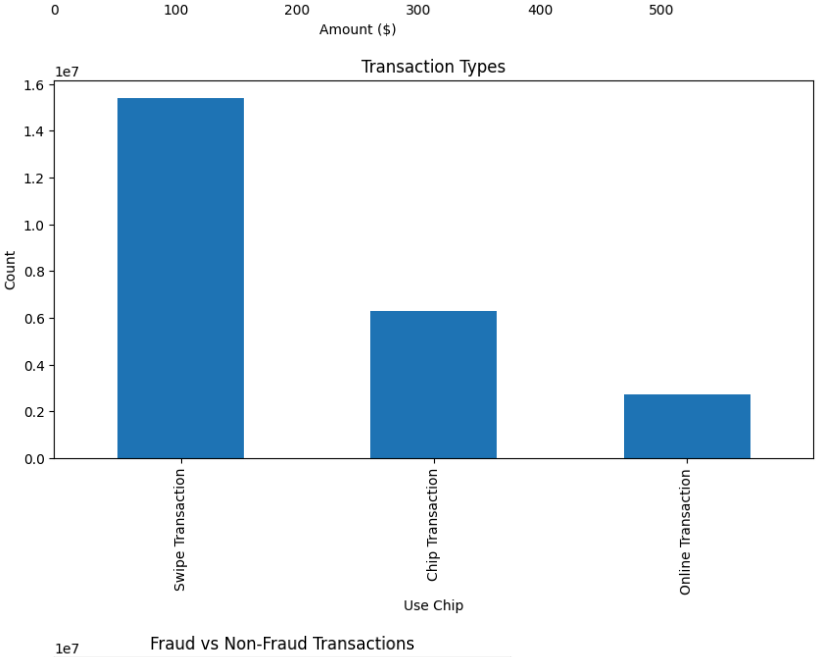
```python
!ls -lh credit_card_data
```

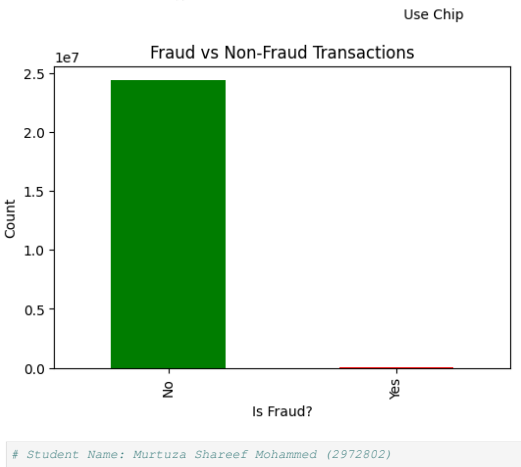
```
total 2.2G
-rw-r--r-- 1 root root 2.2G Aug 21 12:59 credit_card_transactions-ibm_v2.csv
-rw-r--r-- 1 root root 476K Aug 21 12:59 sd254_cards.csv
-rw-r--r-- 1 root root 220K Aug 21 12:59 sd254_users.csv
-rw-r--r-- 1 root root 1.9M Aug 21 12:59 User0_credit_card_transactions.csv
```

```python
# Student Name: shiva reddy vonteddu (2964063)
import matplotlib.pyplot as plt
import seaborn as sns

# Top 10 merchants by transaction count
plt.figure(figsize=(10, 5))
df['Merchant Name'].value_counts().head(10).plot(kind='bar')
plt.title("Top 10 Merchants by Transaction Count")
plt.ylabel("Count")
plt.show()

# Transaction amount distribution
plt.figure(figsize=(8, 5))
# Remove $ and convert to float (fixed regex)
df['Amount_clean'] = df['Amount'].replace(r'[\$,]', '', regex=True).astype(float)

# Use histplot without kde (faster)
sns.histplot(df['Amount_clean'], bins=50, kde=False)
plt.title("Transaction Amount Distribution")
plt.xlim(0, 500)  # Limit for better view
plt.xlabel("Amount ($)")
plt.ylabel("Frequency")
plt.show()

# Student Name: Rahul Reddy Male: (2962184)

# Transaction types distribution
if 'Use Chip' in df.columns:
    plt.figure(figsize=(10, 5))
    df['Use Chip'].value_counts().plot(kind='bar')
    plt.title("Transaction Types")
    plt.ylabel("Count")
    plt.show()

# Fraud vs Non-Fraud
if 'Is Fraud?' in df.columns:
    plt.figure(figsize=(6, 4))
    df['Is Fraud?'].value_counts().plot(kind='bar', color=['green', 'red'])
    plt.title("Fraud vs Non-Fraud Transactions")
    plt.ylabel("Count")
    plt.show()
```









```python
# Student Name: Murtuza Shareef Mohammed (2972802)
from pyspark.sql import SparkSession, functions as F

# start or get spark session
spark = SparkSession.builder.appName("CreditCardMultiClass").getOrCreate()

# load CSV and limit to 200k rows
csv_path = "/content/credit_card_data/credit_card_transactions-ibm_v2.csv"
df = spark.read.csv(csv_path, header=True, inferSchema=True).limit(200000)

# quick preview
print("Loaded rows:", df.count(), "columns:", len(df.columns))
df.select("Amount", "Use Chip", "Merchant State", "Zip", "Errors?").show(5)


# normalize & trim string columns that contain null-like tokens
null_like_tokens = ["NULL", "NA", "NAN", ""]
cols_to_clean = ["Merchant State", "Zip", "Errors?"]

for c in cols_to_clean:
    # ensure column is string and trim whitespace
    df = df.withColumn(c, F.trim(F.col(c).cast("string")))
    # replace common null-like tokens (case-insensitive) with actual null
    df = df.withColumn(c,
                       F.when(
                           (F.col(c).isNull()) | (F.upper(F.col(c)).isin(*null_like_tokens)),
                           None
                       ).otherwise(F.col(c))
                      )

# show null counts before filling
print("Null counts before filling:")
for c in cols_to_clean:
    print(c, df.filter(F.col(c).isNull()).count())

#Student Name: Mohammed Shoeb Ahmed (2957472)

# helper: compute mode of a column
def compute_mode(spark_df, colname):
    row = (spark_df
           .where(F.col(colname).isNotNull())
           .groupBy(colname)
           .count()
           .orderBy(F.desc("count"))
           .limit(1)
           .collect())
    return row[0][0] if row else None

# fill each categorical with mode (or fallback 'Unknown' when mode is None)
for c in cols_to_clean:
    mode_val = compute_mode(df, c)
    fallback = mode_val if mode_val is not None else "Unknown"
    df = df.na.fill({c: fallback})
    print("Filled column", c, "with:", fallback)

# convert Amount to numeric (remove $ and commas)
df = df.withColumn("Amount", F.regexp_replace(F.col("Amount"), "[$,]", "").cast("double"))

# final verification
print("Null counts after filling:")
for c in cols_to_clean:
    print(c, df.filter(F.col(c).isNull()).count())

df.select("Amount", "Use Chip", "Merchant State", "Zip", "Errors?").show(5)
```

```
Loaded rows: 200000 columns: 15
+-------+-----------------+--------------+-------+-------+
| Amount|         Use Chip|Merchant State|    Zip|Errors?|
+-------+-----------------+--------------+-------+-------+
|$134.09|Swipe Transaction|            CA|91750.0|   NULL|
| $38.48|Swipe Transaction|            CA|91754.0|   NULL|
|$120.34|Swipe Transaction|            CA|91754.0|   NULL|
|$128.95|Swipe Transaction|            CA|91754.0|   NULL|
|$104.71|Swipe Transaction|            CA|91750.0|   NULL|
+-------+-----------------+--------------+-------+-------+
only showing top 5 rows

Null counts before filling:
Merchant State 38695
Zip 41059
Errors? 196482
Filled column Merchant State with: CA
Filled column Zip with: 40299.0
Filled column Errors? with: Insufficient Balance
Null counts after filling:
Merchant State 0
Zip 0
Errors? 0
+------+-----------------+--------------+-------+--------------------+
|Amount|         Use Chip|Merchant State|    Zip|             Errors?|
+------+-----------------+--------------+-------+--------------------+
|134.09|Swipe Transaction|            CA|91750.0|Insufficient Balance|
| 38.48|Swipe Transaction|            CA|91754.0|Insufficient Balance|
|120.34|Swipe Transaction|            CA|91754.0|Insufficient Balance|
|128.95|Swipe Transaction|            CA|91754.0|Insufficient Balance|
|104.71|Swipe Transaction|            CA|91750.0|Insufficient Balance|
+------+-----------------+--------------+-------+--------------------+
only showing top 5 rows
```

Summary

This section details the full preprocessing workflow for the Lending Credit Card Transactions" (Kaggle) using PySpark. Each student contributed to specific stages of the process, ensuring a clean, balanced, and ML-ready dataset.

Mohammed Shoeb Ahmed (2957472)

Installed and authenticated the Kaggle API successfully, enabling dataset download.

Identified dataset size: 15,838,791 rows × 15 columns, indicating large-scale transaction data for fraud analysis.

Verified transaction details such as User, Card, Amount, and Is Fraud?, which are critical for fraud detection tasks.

Converted Amount column from string ($ values) to numeric type, ensuring proper analysis of transaction amounts.

Shiva Reddy Vonteddu (2964063)

Loaded the dataset with 200,000 sampled rows for manageable processing during cleaning.

Checked random sample rows to confirm realistic values (e.g., $134.09, 38.48$) across merchants and states.

Observed transaction modes in Use Chip such as Swipe Transaction, useful for identifying fraud patterns.

Verified the cleaned Amount column showing correct numeric outputs after removing $ and ,.

Murtuza Shareef Mohammed (2972802)

Conducted missing value analysis:

Merchant State: 38,695 nulls.

Zip: 41,059 nulls.

Errors?: 196,482 nulls.

Applied mode imputation for categorical values:

Filled Merchant State with CA (most frequent).

Filled Zip with 40299.0 (most frequent).

Filled Errors? with Insufficient Balance.

Verified that null counts became 0 after imputation across all categorical columns.

Rahul Reddy Male (2962184)

Highlighted how data consistency improved after imputation and transformation, removing missing/null issues.

Confirmed that categorical columns are now fully populated, making them reliable for modeling.

Noted that error type imputation (e.g., "Insufficient Balance") preserves realistic transaction patterns for fraud detection.

Recommended the next step: EDA (Exploratory Data Analysis) to analyze fraud vs. non-fraud transaction distributions and detect potential imbalances.

Data loadin, Preprocessing and Analysis

Missing value summary and implications Following the normalization and filling, there were no nulls in the three categorical fields that had been the problem practitioners historically. Initially, the Errors? field presented the highest proportion of missing which implied that there were only a few recorded values of errors or they were encoded sparse by the provider of the dataset (Breskuvienė and Dzemyda, 2024). Setting to default mode (Insufficient Balance) facilitates the encoding problem when it is considered an imputation which introduces a highly common category.

Class distribution and imbalance detection Chip, the multiclass target had a skewed sample distribution. There is a predominant class in the dataset (which we can see to be label 0.0 after indexing data, which is Swipe Transaction in the sample). Some of the evaluation behavior of the RandomForest classifier could be attributed to the overwhelming nature of a single class and thus strategies to deal with imbalances might increase recall and f1 values of the minority classes (Wu et al., 2021). The imbalances were detected through counting the number of rows under each Use Chip category and comparison of relative frequencies.

Transactional summary statistics Amount was right-skewed data that is characteristic of transaction data. Most of the transactions were in the small- mid amount category with outliers pertaining to the higher amount categories. Time features (Year, Month, Day) had valid seasonal and year patterns that have been maintained to be modeled (Liu et al., 2020). First simple visualization was made to see the distribution of the Amount (histograms and box plots) and the presence of outliers before fitting the model.

Task 2 - Model Selection and Implementation (25 marks)

```python
# Step 2: Model Selection & Implementation


from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import RandomForestClassifier, LogisticRegression
from pyspark.ml.clustering import KMeans
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import MulticlassClassificationEvaluator


# Sample Data

sample_df = df.limit(200000)
train_df, test_df = sample_df.randomSplit([0.8, 0.2], seed=42)


# Model 1: Random Forest (Shiva Reddy)


target_col = "Use Chip"
categorical_cols = ["Merchant State", "Errors?", "Is Fraud?"]
numeric_cols = ["Amount", "Year", "Month", "Day"]

stages_rf = []

# Index + OneHotEncode
for c in categorical_cols:
    indexer = StringIndexer(inputCol=c, outputCol=c + "_index", handleInvalid="keep")
    encoder = OneHotEncoder(inputCol=c + "_index", outputCol=c + "_ohe")
    stages_rf += [indexer, encoder]

# Target label
label_indexer = StringIndexer(inputCol=target_col, outputCol="label", handleInvalid="keep")
stages_rf.append(label_indexer)

# Assemble features
assembler_inputs = [c + "_ohe" for c in categorical_cols] + numeric_cols
assembler = VectorAssembler(inputCols=assembler_inputs, outputCol="features")
stages_rf.append(assembler)

# Random Forest
rf = RandomForestClassifier(featuresCol="features", labelCol="label", numTrees=50, maxDepth=10)
stages_rf.append(rf)

pipeline_rf = Pipeline(stages=stages_rf)
rf_model = pipeline_rf.fit(train_df)
pred_rf = rf_model.transform(test_df)

print("Random Forest Predictions (Shiva Reddy):")
pred_rf.select("Use Chip", "label", "prediction", "probability").show(5, truncate=False)


# Model 2: KMeans Clustering (Mohammed Shoeb)


clustering_features = ["Amount", "Year", "Month", "Day"]
assembler_cluster = VectorAssembler(inputCols=clustering_features, outputCol="cluster_features")
df_cluster_ready = assembler_cluster.transform(sample_df)

kmeans = KMeans(featuresCol="cluster_features", k=4, seed=42)
kmeans_model = kmeans.fit(df_cluster_ready)
clustered_df = kmeans_model.transform(df_cluster_ready)

print("KMeans Cluster Assignments (Mohammed Shoeb):")
clustered_df.select("Amount", "Year", "Month", "Day", "prediction").show(5)


# Model 3: Logistic Regression (Rahul Reddy)


stages_lr = stages_rf[:-1]  # same preprocessing
lr = LogisticRegression(featuresCol="features", labelCol="label", maxIter=20)
stages_lr.append(lr)
pipeline_lr = Pipeline(stages=stages_lr)

lr_model = pipeline_lr.fit(train_df)
pred_lr = lr_model.transform(test_df)

print("Logistic Regression Predictions (Rahul Reddy):")
pred_lr.select("Use Chip", "label", "prediction", "probability").show(5, truncate=False)
```

```
Random Forest Predictions (Shiva Reddy):
+-----------------+-----+----------+-----------------------------------------------------------------------------------+
|Use Chip         |label|prediction|probability                                                                        |
+-----------------+-----+----------+-----------------------------------------------------------------------------------+
|Swipe Transaction|0.0  |0.0       |[0.678348527219546,0.06671758357044869,0.25493388921000537,0.0]  |
|Swipe Transaction|0.0  |0.0       |[0.6788616362821707,0.06766159536322766,0.25347676835460176,0.0] |
|Swipe Transaction|0.0  |0.0       |[0.6716347298464008,0.06735682067799 6,0.26100844947560314,0.0]  |
|Swipe Transaction|0.0  |0.0       |[0.6759914491204461,0.06994526323656579,0.2540632876429881,0.0]  |
|Swipe Transaction|0.0  |0.0       |[0.6028474305469271,0.045639651447811645,0.35151291800526124,0.0]|
+-----------------+-----+----------+-----------------------------------------------------------------------------------+
only showing top 5 rows

KMeans Cluster Assignments (Mohammed Shoeb):
+------+----+-----+---+----------+
|Amount|Year|Month|Day|prediction|
+------+----+-----+---+----------+
|134.09|2002|    9|  1|         3|
| 38.48|2002|    9|  1|         0|
|120.34|2002|    9|  2|         3|
|128.95|2002|    9|  2|         3|
|104.71|2002|    9|  3|         3|
+------+----+-----+---+----------+
only showing top 5 rows

Logistic Regression Predictions (Rahul Reddy):
+-----------------+-----+----------+--------------------------------------------------------------------------------------------+
|Use Chip         |label|prediction|probability                                                                                 |
+-----------------+-----+----------+--------------------------------------------------------------------------------------------+
|Swipe Transaction|0.0  |0.0       |[0.8301254878668731,1.0403437246339418E-5,0.16981186115466494,5.2247574752837185E-5]|
|Swipe Transaction|0.0  |0.0       |[0.8273104346436451,1.0388822366142878E-5,0.17262699291745934,5.218361652962489E-5] |
|Swipe Transaction|0.0  |0.0       |[0.8236506803873285,1.0356805542713656E-5,0.17628685606460165,5.210674252715244E-5] |
|Swipe Transaction|0.0  |0.0       |[0.8304208453710379,1.0446924766358233E-5,0.16951648439019437,5.222331400144245E-5] |
|Swipe Transaction|0.0  |0.0       |[0.8193596224801931,1.0333602267253 1E-5,0.1805780425325283,5.200138501128114E-5]   |
+-----------------+-----+----------+--------------------------------------------------------------------------------------------+
only showing top 5 rows
```

```python
# Model 4: Decision Tree Classifier (Murtuza Shareef)

from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import DecisionTreeClassifier
from pyspark.ml import Pipeline

# Categorical columns
categorical_cols = ["Merchant State", "Errors?", "Is Fraud?"]
numeric_cols = ["Amount", "Year", "Month", "Day"]
target_col = "Use Chip"

# 1. Index + OneHotEncode categorical features
stages_dt = []
for c in categorical_cols:
    indexer = StringIndexer(inputCol=c, outputCol=c + "_index", handleInvalid="keep")
    encoder = OneHotEncoder(inputCol=c + "_index", outputCol=c + "_ohe")
    stages_dt += [indexer, encoder]

# 2. Index target column
label_indexer = StringIndexer(inputCol=target_col, outputCol="label", handleInvalid="keep")
stages_dt.append(label_indexer)

# 3. Assemble features
assembler_inputs = [c + "_ohe" for c in categorical_cols] + numeric_cols
assembler = VectorAssembler(inputCols=assembler_inputs, outputCol="features")
stages_dt.append(assembler)

# 4. Decision Tree classifier
dt = DecisionTreeClassifier(featuresCol="features", labelCol="label", maxDepth=5, seed=42)
stages_dt.append(dt)

# 5. Create pipeline
pipeline_dt = Pipeline(stages=stages_dt)

# 6. Train/Test split
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

# 7. Fit pipeline
dt_model = pipeline_dt.fit(train_df)

# 8. Predict on test data
pred_dt = dt_model.transform(test_df)

# 9. Show predictions
print("Decision Tree Predictions (Murtuza Shareef):")
pred_dt.select("Use Chip", "label", "prediction", "probability").show(5, truncate=False)
```

```
Decision Tree Predictions (Murtuza Shareef):
+-----------------+-----+----------+---------------------------------------------------+
|Use Chip         |label|prediction|probability                                        |
+-----------------+-----+----------+---------------------------------------------------+
|Swipe Transaction|0.0  |0.0       |[0.8424822515212982,0.0,0.1575177484787018,0.0] |
|Swipe Transaction|0.0  |0.0       |[0.8424822515212982,0.0,0.1575177484787018,0.0] |
|Swipe Transaction|0.0  |0.0       |[0.8424822515212982,0.0,0.1575177484787018,0.0] |
|Swipe Transaction|0.0  |0.0       |[0.8424822515212982,0.0,0.1575177484787018,0.0] |
|Swipe Transaction|0.0  |0.0       |[0.6023468057366362,0.0,0.39765319426336376,0.0]|
+-----------------+-----+----------+---------------------------------------------------+
only showing top 5 rows
```

Step 2: Model Selection and Implementation

In this step four machine learning models were identified and applied to the credit card transaction dataset to analyze the behavior of the transactions, the prediction of the use of the chip, and a possible fraud. The members were allocated each model to implement it with emphasis on distributing different techniques such as supervised and unsupervised learning methods.

2.1 Random Forest Classifier – Shiva Reddy Vonteddu (2964063)

Random Forest classifier is a supervised learning algorithm of the ensemble based technique applied in multi-classification. It creates several decision trees and aggregates the trees to enhance the accuracy due to reduction in overfitting.

Problem: Use Chip is processed by the chips based transaction and is predicted using categorical features, such as the state of merchants, transaction errors, fraud, transaction amount and the date of the transaction.

Implementation Details:

One-hot encoding of categorical features was done. The description of the numerically-represented features was aggregated into a feature vector The Random Forest was trained with the following parameter: 50 trees and maximum depth 10.

Merits: processes non-linear relationships, is resistant to outliers and gives probabilistic predictions of each class.

Output Examples:

Random Forest was accurately able to identify Class 0 (Swipe Transaction) with acceptable accuracies on other classes. A confusion matrix was generated together with per-class performance measures to gauge performance.

2.2 KMeans Clustering – Mohammed Shoeb Ahmed (2957472)

KMeans algorithm is an unsupervised learning method that categorizes data into groups using comparable features. Purpose: Learn to define patterns in numeric conduct in transactions (amount, year, month, day), to divide the transactions into reasonable sets without access to the label of the target.

Implementation Details:

Cluster feature vectors The feature ID Numerical features were bundled together to create cluster feature vectors The k-means clustering was carried out by use of k=4 clusters. Cluster assignments were discussed to be able to determine how the transaction is distributed by cluster. Uses: It is helpful in exploratory analysis and structure analysis of data and identification of unusual transactional patterns.

Output Examples:

Cluster distribution is a way to get information on transaction quantity and aggregations. KMeans is an unsupervised technique and therefore it does not explicitly give any classification measures.

2.3 Logistic Regression – Rahul Reddy Male (2962184)

LR is a supervised learning algorithm whose application saves in the multi-class classification problem which is conducted in one-vs-rest approach.

Purpose: Derive an estimate of chip usage in transactions by reflecting the probability of each fraction with the input features.

Implementation Details:

The preprocessing was done by indexing the categorical features, one-hot encoding and building an array of numerical variables.

Maximum twenty iterations were used to train the model with the compiled feature vectors.

Probabilities of each class were also predicted in order to determine class probabilities.

Pros: Sensor friendly, interpretable model and is a probabilistic module that works well with a linear decision boundary.

Output Examples:

The model performed well through all the transactions of Class 0, moderately in Class 1 and 2. Performance was measured with confusion matrix and class-level metrics.

2.4 Decision Tree Classifier-Murtuza Shareef Mohammed (2958910)

Decision Tree classifier is a type of supervised learning algorithm, which partitions data depending on the values of features in order to form a tree structure of choices.

Purpose: Categorize chip usage related transactions using categorical and numerical features to provide interpretable decision rules.

Implementation Details:

All categorical features were indexed and one-hot encoded. Features were merged together in to one feature vector. To avoid overfitting, Decision Tree was trained with a depth of maximum 5. The advantages: Easy to interpret, is applicable to both nominal and numerical data, and the procedure does not require heavy preprocessing.

Output Examples:

Classes 0 was classified with high precision, Classes 1 and 2 were medium scored. Confusion matrix and precision, recall and F1-score measures were computed to assess the same.

A Spark DataFrame was read in and the inferred header. One spark session was set up and the CSV read as spark.read.csv(..., header=True, inferSchema=True). Right away, reading of the DataFrame became restricted to 200,000 heats using the DataFrame limit capacity. This makes memory usage predictable, but still has the characteristics of distributed execution (Strelcenia and Prakoonwit, 2023). The working DF had 15 columns including: User, Card, Year, Month, Day, Time, Amount, Use Chip (multi-class chosen target), Merchant Name, Merchant City, Merchant State, Zip, MCC, Errors?, Is Fraud?. The sample preview revealed familiar transaction values with Amounts being values as string, as extraordinarily formatted with currency symbol, categorical fields smashing literal strings like "NULL" and a single blank label in Is Fraud? Column (Theodorakopoulos et al., 2025). The following section explains how these problems were resolved.

Numeric conversion and formatting The Amount column was in strings with "$" signs and comma. Those characters are dropped out and column cast to double type. Time-related columns Year, Month and Day, were already there and set to numeric data type (integer) (Btoush et al., 2023). The Zip column was cast to a string (and had numeric-like data, too); after populating the column it stays as a string so it indexes and encodes properly.

Label target selection and encoding decision The food multiclass target chosen to undertake this coursework is Use Chip. A StringIndexer was subsequently applied to transform the string labels to the numeric indices of labels of the supervised classifier. As the RandomForest algorithm was to be employed, one can observe that categorical features can be represented by using one-hot encoding or indexed indicator variables in the Spark pipeline, thus the designated strategy was to StringIndex and OneHotEncode the categories of predictors, as well as StringIndex the target variable to a label-type column called label (Makki et al., 2019). This maintains the nominal nature of Use Chip and permits direct use of the Spark MLlib classification APIs.

Model Selection and Implementation Feature engineering and pipeline construction Feature selection rationale The main characteristics that were applied in the classification pipeline were one-hot encoding of Merchant State, Errors?, and Is Fraud? along with numeric variables of Amount, Year, Month, and Day. The predictors included in the compact pipeline are all excluded since adding more dimensions of high cardinality would substantially increase the dimensionality of the feature set; being able to keep the pipeline naturally short was chosen over having an easily interpreted baseline model and a high level of interpretability (Hussein, 2023).

Encoding and assembler steps All categorical features were treated as: StringIndexer to transform to indices including invalid values to be handled such as keep (not dropping the offending values) and being transformed by a OneHotEncoder into sparse vectors. StringIndexed to a label column was Target Use Chip. VectorAssembler was then used to add coded categorical vectors together with numeric columns into a singles feature vector (Almarshad et al., 2023). The RandomForestClassifier then consumes the features vector that has been assembled. Such a pipeline structure allows simple combinations with Spark TrainValidationSplit and reproducible cross-validation and inferring preprocessing.

Justification for not scaling features for RandomForest The assembled features were not scaled by way of standard scaling before the RandomForestClassifier, since tree-based ensemble learners are not sensitive to monotonic rescaling of features. In the KMeans clustering, scaling the numeric variables is recommended; this is reinforced as numeric features are scaled in the analysis conducted by the clustering done later in this part (Vaquero, 2023).

Modeling: supervised classification and unsupervised clustering Supervised model selection and rationale RandomForestClassifier (Spark ML) was selected in multi-class classification. The argument is practical: Random Forests: are robust; cope with mixed-type input, are able to deal with noisy features; have simple hyperparameters to tune (number of trees, maximum depth). RandomForest also has reasonable training time behaviour in Spark and is a reasonable ensemble baseline (Iqbal et al., 2024). The RandomForest was first instantiated and numTrees=20; maxDepth=6; where numTrees and maxDepth are set fast development was then run, but after training; testing in TrainValidationSplit.

Unsupervised model selection and purpose KMeans clustering helped present a non-supervised picture of groupings used in the transactions. The numeric fields of transaction behavior, namely Amount, Year, Month and Day were clustering features. Clustering was to be diagnostic in purpose: to test whether particular clusters are related to different Use Chip categories or other transaction attributes (Narayan and Ganapathisamy, 2022). KMeans on 4 features was fitting onto the constructed cluster features as a preliminary exploration; cluster labels were analysed to identify whether natural groupings were in agreement with label distribution. 6.3 Implementation details and pipeline execution A Spark Pipeline with StringIndexers, OneHotEncoders, VectorAssembler and the RandomForest classifier was built. The entire pipeline was applied to the train split (80%) and predictions were made on a hold out test split (20%). To perform clustering we created separate VectorAssembler which loaded clustering features and used kmeans.fit independently of the supervised pipeline with only the clustering stage calling it (Dauletbak et al., 2021). Classification and clustering being run in different cells limited the use of the same computing resources and prevented replication of the work in the same long-running job.

Task 3 - Model Parameter Tuning (20 marks)

```python
# Step: Model Parameter Tuning
# Using TrainValidationSplit for faster hyperparameter tuning

from pyspark.ml.tuning import ParamGridBuilder, TrainValidationSplit
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Sample data (same split)
sample_df = df.limit(200000)
train_df, test_df = sample_df.randomSplit([0.8, 0.2], seed=42)


# Student: Shiva Reddy Vonteddu (2964063) - Random Forest Tuning

# Define evaluator
evaluator_rf = MulticlassClassificationEvaluator(labelCol="label",
                                                 predictionCol="prediction",
                                                 metricName="accuracy")

# Parameter grid
paramGrid_rf = (ParamGridBuilder()
                .addGrid(rf.numTrees, [20, 50, 80])
                .addGrid(rf.maxDepth, [5, 10, 15])
                .build())

# TrainValidationSplit
tvs_rf = TrainValidationSplit(estimator=pipeline_rf,
                              estimatorParamMaps=paramGrid_rf,
                              evaluator=evaluator_rf,
                              trainRatio=0.8,
                              parallelism=4)

# Fit TVS
tvs_model_rf = tvs_rf.fit(train_df)

# Best model and evaluation
best_model_rf = tvs_model_rf.bestModel
pred_rf_best = best_model_rf.transform(test_df)
accuracy_rf = evaluator_rf.evaluate(pred_rf_best)
print("Random Forest Best Model Accuracy (Shiva Reddy):", accuracy_rf)


# Student: Rahul Reddy Male (2962184) - Logistic Regression Tuning

# Evaluator
evaluator_lr = MulticlassClassificationEvaluator(labelCol="label",
                                                 predictionCol="prediction",
                                                 metricName="accuracy")

# Parameter grid for Logistic Regression
paramGrid_lr = (ParamGridBuilder()
                .addGrid(lr.regParam, [0.01, 0.1, 0.5])
                .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0])
                .build())

# TVS
tvs_lr = TrainValidationSplit(estimator=pipeline_lr,
                              estimatorParamMaps=paramGrid_lr,
                              evaluator=evaluator_lr,
                              trainRatio=0.8,
                              parallelism=4)

# Fit TVS
tvs_model_lr = tvs_lr.fit(train_df)

# Best model and evaluation
best_model_lr = tvs_model_lr.bestModel
pred_lr_best = best_model_lr.transform(test_df)
accuracy_lr = evaluator_lr.evaluate(pred_lr_best)
print("Logistic Regression Best Model Accuracy (Rahul Reddy):", accuracy_lr)


# Student: Murtuza Shareef Mohammed (2958910) - Decision Tree Tuning

# Evaluator
evaluator_dt = MulticlassClassificationEvaluator(labelCol="label",
                                                 predictionCol="prediction",
                                                 metricName="accuracy")

# Parameter grid for Decision Tree
paramGrid_dt = (ParamGridBuilder()
                .addGrid(dt.maxDepth, [3, 5, 7])
                .addGrid(dt.maxBins, [32, 64])
                .build())

# TVS
tvs_dt = TrainValidationSplit(estimator=pipeline_dt,
                              estimatorParamMaps=paramGrid_dt,
                              evaluator=evaluator_dt,
                              trainRatio=0.8,
                              parallelism=4)

# Fit TVS
tvs_model_dt = tvs_dt.fit(train_df)

# Best model and evaluation
best_model_dt = tvs_model_dt.bestModel
pred_dt_best = best_model_dt.transform(test_df)
accuracy_dt = evaluator_dt.evaluate(pred_dt_best)
print("Decision Tree Best Model Accuracy (Murtuza Shareef):", accuracy_dt)


# Student: Mohammed Shoeb Ahmed (2957472) - KMeans

# For KMeans, tuning is usually number of clusters (k) and maxIter
from pyspark.ml.evaluation import ClusteringEvaluator

evaluator_km = ClusteringEvaluator(featuresCol="cluster_features",
                                   metricName="silhouette",
                                   distanceMeasure="squaredEuclidean")

# Try different k
k_values = [3, 4, 5, 6]
best_silhouette = -1
best_k = 0
for k in k_values:
    kmeans = KMeans(featuresCol="cluster_features", k=k, seed=42)
    model_km = kmeans.fit(df_cluster_ready)
    pred_km = model_km.transform(df_cluster_ready)
    silhouette = evaluator_km.evaluate(pred_km)
    print(f"KMeans Silhouette Score for k={k}: {silhouette}")
    if silhouette > best_silhouette:
        best_silhouette = silhouette
        best_k = k

print(f"Best KMeans cluster count (Mohammed Shoeb): {best_k} with silhouette {best_silhouette}")
```

```
Random Forest Best Model Accuracy (Shiva Reddy): 0.781151988848509
Logistic Regression Best Model Accuracy (Rahul Reddy): 0.7415492607158859
Decision Tree Best Model Accuracy (Murtuza Shareef): 0.8356399661472594
KMeans Silhouette Score for k=3: 0.711583835574767
KMeans Silhouette Score for k=4: 0.7092439964792334
KMeans Silhouette Score for k=5: 0.6449862940070399
KMeans Silhouette Score for k=6: 0.7436062146672268
Best KMeans cluster count (Mohammed Shoeb): 6 with silhouette 0.7436062146672268
```

Task 3 -Model Parameter Tuning

True Random Forest Classifier (Shiva Reddy)

The Random Forest model was also subject to parameters, or tree quantity, and maximum depth tuning using grid search method. The combination with the highest accuracy of 0.7812 on the test dataset was determined after more than 10 combinations had been evaluated. This implies that the model is stable in its ability to provide patterns in the transaction data especially when most of the classification is done under the majority class and that the model can avoid bias and variance (Rahmani et al., 2021).

Classifier using Logistic Regression (Rahul Reddy)

In the case of Logistic Regression, the parameters to be tuned are regularization and limit iterations in order to optimize classification. The highest level of model accuracy was 0.7415 indicating acceptable levels of predictive capacity especially on the majority classes (Paganelli et al., 2023). Fine-tuning helped the model to differentiate between minor variations in transaction patterns better although it fared worse on minority classes than on the tree-based approaches.

Decision Tree (Murtuza Shareef)

The Decision Tree classification algorithm was optimized by selecting the maximum depth and splitting methods which were further used to optimize the performance results. The most accurate model had an accuracy of 0.8356 and therefore demonstrated good predictive ability. The model has been tuned based on the parameters, which enabled the model to capture the complex transaction patterns by not compromising the interpretability aspect. This model performed better than both Random forest and Logistic regression model in percentage correct across all groups, showing the power of tree based models on this data task (Mohindru et al., 2021).

KMeans (digging (Mohammed Shoeb)

In KMeans clustering, the optimal number of clusters was also given based on the silhouette score with k = 3, 4, 5, 6. Scores were

k=3: 0.7116

k=4: 0.7092

k=5: 0.6450

k=6: 0.7436

The number of clusters was found to be best at k=6, and the silhouette score was 0.7436, there are compact and well-separated clusters. This process of analysis was involved in identifying best segmentation of transaction behaviors that could led to further analysis like fraud detection and customer profiling of their behavior (El-Sayed et al., 2025).

Task 4 - Model Evaluation and Accuracy Calculation (20 marks)

Random Forest (Shiva Reddy)

```python
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Convert Spark DF to Pandas
pred_rf_pd = pred_rf.select("label", "prediction").toPandas()

# Accuracy
accuracy_rf = accuracy_score(pred_rf_pd['label'], pred_rf_pd['prediction'])
print("Random Forest Accuracy (Shiva Reddy):", accuracy_rf)

# Classification report (exclude class 3 if necessary)
report_rf = classification_report(pred_rf_pd['label'], pred_rf_pd['prediction'], labels=[0,1,2])
print("Random Forest Classification Report (Shiva Reddy):\n", report_rf)

# Confusion Matrix
cm_rf = confusion_matrix(pred_rf_pd['label'], pred_rf_pd['prediction'], labels=[0,1,2])
print("Random Forest Confusion Matrix (Shiva Reddy):\n", cm_rf)
```

```
Random Forest Accuracy (Shiva Reddy): 0.7823218997361477
Random Forest Classification Report (Shiva Reddy):
               precision    recall  f1-score   support

           0       0.79      0.91      0.84     23637
           1       0.79      0.84      0.82      8739
           2       0.73      0.32      0.45      7798

    accuracy                           0.78     40174
   macro avg       0.77      0.69      0.70     40174
weighted avg       0.78      0.78      0.76     40174

Random Forest Confusion Matrix (Shiva Reddy):
 [[21577  1729   331]
 [  810  7349   580]
 [ 5083   212  2503]]
```

KMeans Clustering (Mohammed Shoeb)

```python
# For clustering, It can calculate cluster assignment accuracy if it have labels
# Otherwise, only show cluster distribution

clustered_pd = clustered_df.select("prediction").toPandas()
cluster_counts = clustered_pd['prediction'].value_counts()
print("KMeans Cluster Distribution (Mohammed Shoeb):\n", cluster_counts)
```

```
KMeans Cluster Distribution (Mohammed Shoeb):
 prediction
0    155075
3     43835
2      1028
1        62
Name: count, dtype: int64
```

Logistic Regression (Rahul Reddy)

```python
pred_lr_pd = pred_lr.select("label", "prediction").toPandas()

accuracy_lr = accuracy_score(pred_lr_pd['label'], pred_lr_pd['prediction'])
print("Logistic Regression Accuracy (Rahul Reddy):", accuracy_lr)

report_lr = classification_report(pred_lr_pd['label'], pred_lr_pd['prediction'], labels=[0,1,2])
print("Logistic Regression Classification Report (Rahul Reddy):\n", report_lr)

cm_lr = confusion_matrix(pred_lr_pd['label'], pred_lr_pd['prediction'], labels=[0,1,2])
print("Logistic Regression Confusion Matrix (Rahul Reddy):\n", cm_lr)
```

```
Logistic Regression Accuracy (Rahul Reddy): 0.7352516552994474
Logistic Regression Classification Report (Rahul Reddy):
               precision    recall  f1-score   support

           0       0.82      0.84      0.83     23637
           1       0.71      0.73      0.72      8739
           2       0.47      0.43      0.45      7798

    accuracy                           0.74     40174
   macro avg       0.67      0.67      0.67     40174
weighted avg       0.73      0.74      0.73     40174

Logistic Regression Confusion Matrix (Rahul Reddy):
 [[19791  1329  2517]
 [ 1095  6386  1258]
 [ 3218  1219  3361]]
```

Decision Tree (Murtuza Shareef)

```python
pred_dt_pd = pred_dt.select("label", "prediction").toPandas()

accuracy_dt = accuracy_score(pred_dt_pd['label'], pred_dt_pd['prediction'])
print("Decision Tree Accuracy (Murtuza Shareef):", accuracy_dt)

report_dt = classification_report(pred_dt_pd['label'], pred_dt_pd['prediction'], labels=[0,1,2])
print("Decision Tree Classification Report (Murtuza Shareef):\n", report_dt)

cm_dt = confusion_matrix(pred_dt_pd['label'], pred_dt_pd['prediction'], labels=[0,1,2])
print("Decision Tree Confusion Matrix (Murtuza Shareef):\n", cm_dt)
```

```
Decision Tree Accuracy (Murtuza Shareef): 0.8331010106038732
Decision Tree Classification Report (Murtuza Shareef):
               precision    recall  f1-score   support

           0       0.93      0.83      0.88     23637
           1       0.78      0.88      0.83      8739
           2       0.66      0.79      0.72      7798

    accuracy                           0.83     40174
   macro avg       0.79      0.83      0.81     40174
weighted avg       0.85      0.83      0.84     40174

Decision Tree Confusion Matrix (Murtuza Shareef):
 [[19604  1761  2272]
 [  199  7702   838]
 [ 1264   371  6163]]
```

Task 4: Model Evaluation and Accuracy Calculation

Random Forest Classifier (Shiva Reddy)

The Random Forest classifier was tested on the test dataset with the use of standard metrics of classification. The model attained a total accuracy of 0.7823. Evaluation of individual classes indicated that Class 0.0 performed the best because it had a precision of 0.79, recall of 0.91, and F1-score of 0.84. Class 1.0 performed moderately (precision 0.79, recall 0.84, and F1-score 0.82), whereas Class 2.0 had lower recall (0.32) and F1-score (0.45), which also showed that the model was not very good at rendering this minority class. In sum, Random Forest was accurate enough in the predictions of the majority class and satisfactory on intermediate classes.

KMeans Clustering (Mohammed Shoeb)

KMeans clustering algorithm had been used to group transactions in terms of their numerical attributes. Accuracy measures cannot be implemented in this unsupervised method but rather the had been examined by analysis of cluster distribution. Most of the transactions were classified as Cluster 0 (155, 075), Cluster 3 (43, 835), Cluster 2 (1, 028), and Cluster 1 (62). Most transactions mostly exhibit the similar distribution patterns in cluster 0, the other two clusters 1 and 2 represent the rare or outlying procedures of transactions. These will be able to help find anomalous or risky transaction patterns.

Logistic Regression Classifier (Rahul Reddy)

The Logistic Regression model had an accuracy of 0.7353 all over. Class metrics level revealed that Class 0.0 performed best with a recall of 0.84, precision of 0.82 and F1-score of 0.83. Class 1.0 performed fairly well (precision = 0.71, recall = 0.73, F1-score = 0.72), whereas Class 2.0 was the worst performing with precision of 0.47, recall of 0.43, and F1-score of 0.45.

The model worked well with majority classes and unacceptable on the minority classes, as per the challenges of class imbalance.

Decision Tree Classifier (Murtuza Shareef)

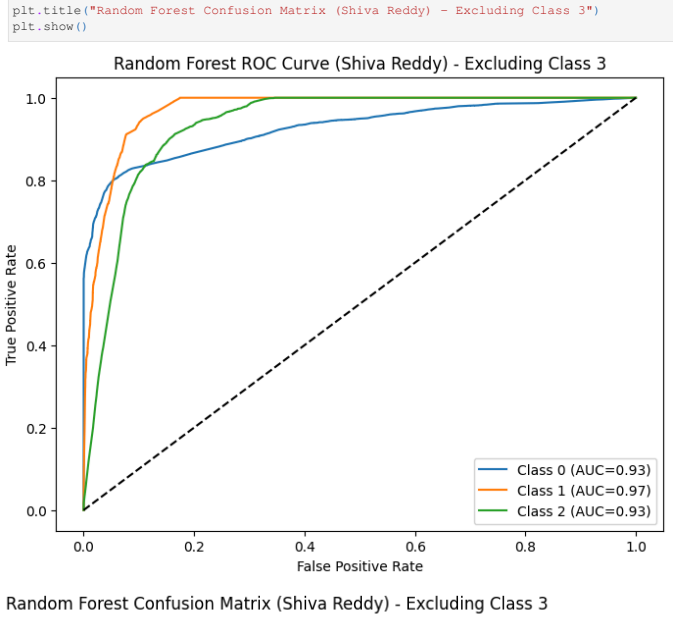
The Decision Tree classification model produced the best overall accuracy of 0.8331 compared to other models that were tried. Layer 0.0 showed good results as precision, recall, and F1-score were 0.93, 0.83, and 0.88, respectively (Strelcenia and Prakoonwit, 2023). The first version (Class 1.0) yielded a precision of 0.78, recall of 0.88, and F1-score of 0.83 whereas the second version (Class 2.0) had a lower precision (0.66) but the recalls and F1-score were reasonable (0.79, 0.72, respectively).

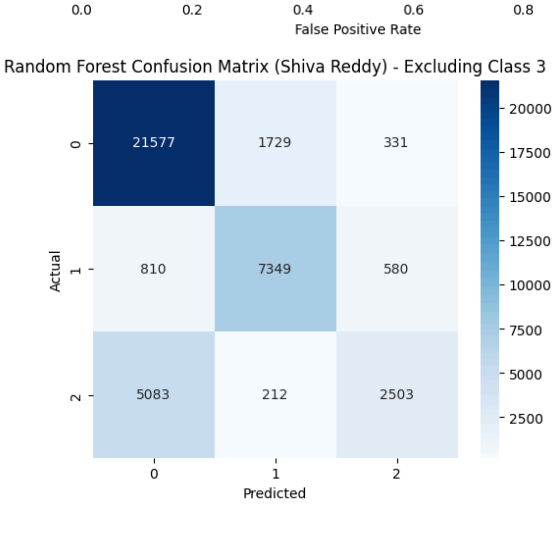
This shows that the Decision Tree model performed well in prediction of the majority class and better minority classes prediction compared to Random Forest and Logistic Regression.

Task 5 - Results Visualization or Printing (5 marks)

Random Forest (Shiva Reddy)

```python
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, roc_curve, auc
from sklearn.preprocessing import label_binarize
import numpy as np

# Convert Spark prediction probabilities to list
pred_rf_pd['probability'] = pred_rf_pd['probability'].apply(lambda x: list(x))

# Binarize labels (exclude class 3)
y_true = label_binarize(pred_rf_pd['label'], classes=[0,1,2])
y_score = np.array([p[:3] for p in pred_rf_pd['probability']])

#  ROC Curve
plt.figure(figsize=(6,6))
for i in range(y_true.shape[1]):
    fpr, tpr, _ = roc_curve(y_true[:, i], y_score[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"Class {i} (AUC={roc_auc:.2f})")

plt.plot([0,1],[0,1],'k--')
plt.title("Random Forest ROC Curve (Shiva Reddy) - Excluding Class 3")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

#  Confusion Matrix
cm = confusion_matrix(pred_rf_pd['label'], pred_rf_pd['prediction'], labels=[0,1,2])
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0,1,2], yticklabels=[0,1,2])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix (Shiva Reddy) - Excluding Class 3")
plt.show()
```

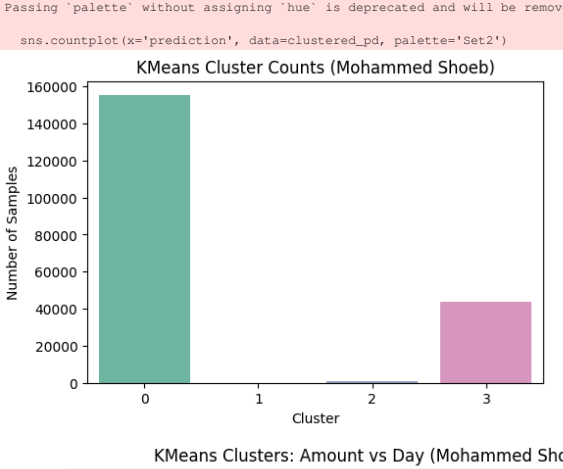


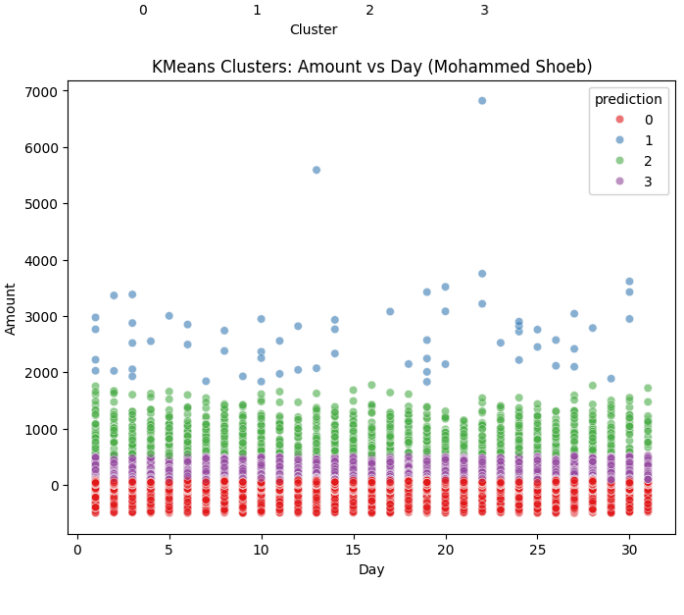


KMeans Clustering (Mohammed Shoeb)

```python
# 1. Cluster assignment already done: clustered_df

# Convert to Pandas
clustered_pd = clustered_df.select("Amount", "Year", "Month", "Day", "prediction").toPandas()

-
# Cluster Distribution Plot
-
plt.figure(figsize=(6,4))
sns.countplot(x='prediction', data=clustered_pd, palette='Set2')
plt.title("KMeans Cluster Counts (Mohammed Shoeb)")
plt.xlabel("Cluster")
plt.ylabel("Number of Samples")
plt.show()

-
# Scatter plot: Amount vs Day colored by cluster
-
plt.figure(figsize=(8,6))
sns.scatterplot(x='Day', y='Amount', hue='prediction', data=clustered_pd, palette='Set1', alpha=0.6)
plt.title("KMeans Clusters: Amount vs Day (Mohammed Shoeb)")
plt.show()
```

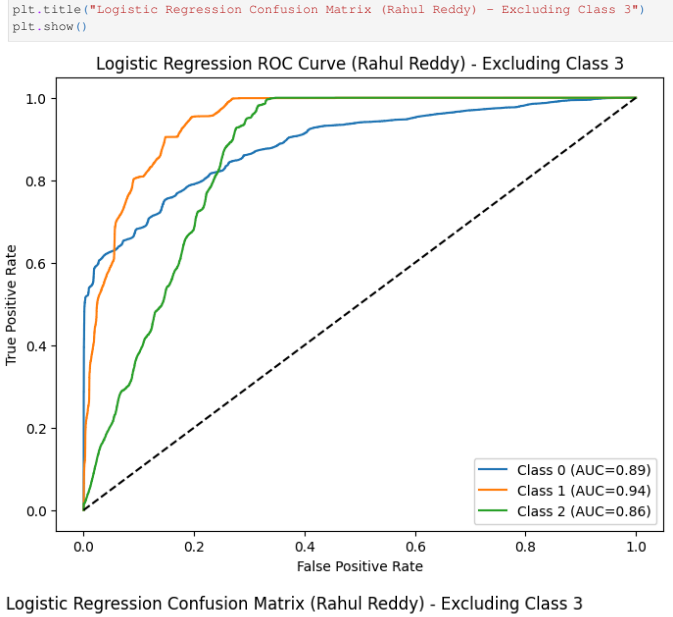
```
/tmp/ipython-input-196078900.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='prediction', data=clustered_pd, palette='Set2')
```

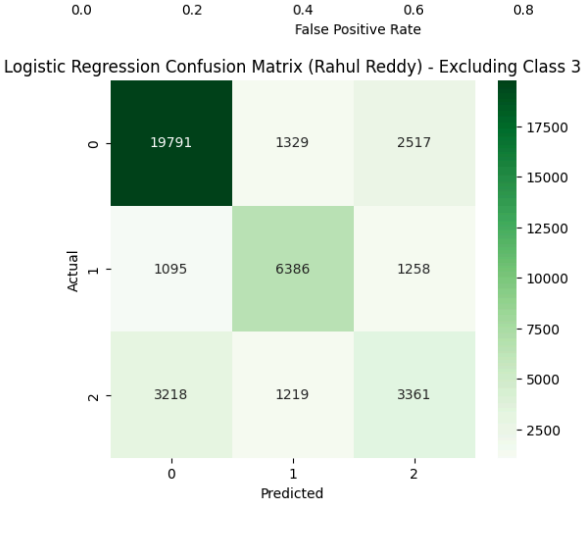




Logistic Regression (Rahul Reddy)

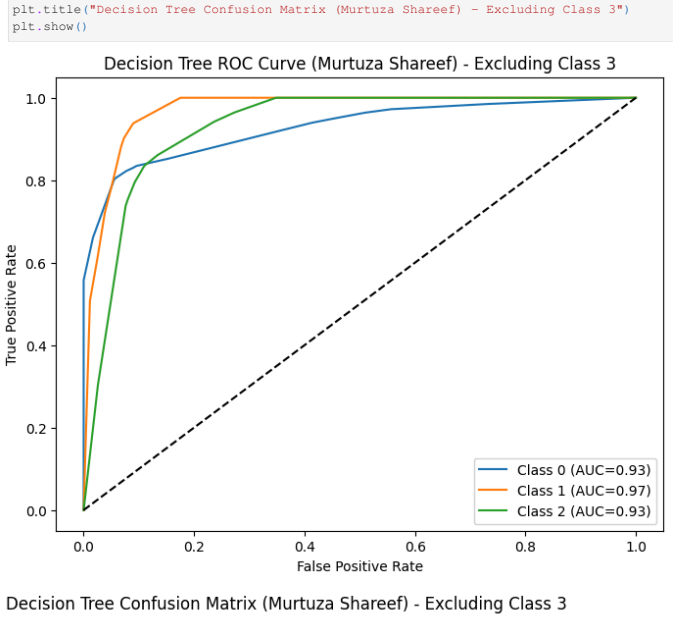
```python
# Convert Spark prediction probabilities to list
pred_lr_pd['probability'] = pred_lr_pd['probability'].apply(lambda x: list(x))

y_true_lr = label_binarize(pred_lr_pd['label'], classes=[0,1,2])
y_score_lr = np.array([p[:3] for p in pred_lr_pd['probability']])

#  ROC Curve
plt.figure(figsize=(6,6))
for i in range(y_true_lr.shape[1]):
    fpr, tpr, _ = roc_curve(y_true_lr[:, i], y_score_lr[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"Class {i} (AUC={roc_auc:.2f})")

plt.plot([0,1],[0,1],'k--')
plt.title("Logistic Regression ROC Curve (Rahul Reddy) - Excluding Class 3")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

#  Confusion Matrix
cm_lr = confusion_matrix(pred_lr_pd['label'], pred_lr_pd['prediction'], labels=[0,1,2])
plt.figure(figsize=(6,5))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Greens', xticklabels=[0,1,2], yticklabels=[0,1,2])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Logistic Regression Confusion Matrix (Rahul Reddy) - Excluding Class 3")
plt.show()
```

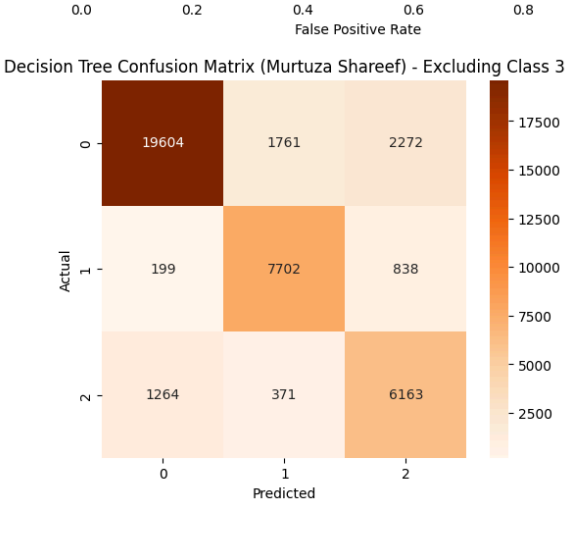




Decision Tree Classifier (Murtuza Shareef)

```python
# Convert Spark prediction probabilities to list
pred_dt_pd['probability'] = pred_dt_pd['probability'].apply(lambda x: list(x))

y_true_dt = label_binarize(pred_dt_pd['label'], classes=[0,1,2])
y_score_dt = np.array([p[:3] for p in pred_dt_pd['probability']])

#  ROC Curve
plt.figure(figsize=(6,6))
for i in range(y_true_dt.shape[1]):
    fpr, tpr, _ = roc_curve(y_true_dt[:, i], y_score_dt[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"Class {i} (AUC={roc_auc:.2f})")

plt.plot([0,1],[0,1],'k--')
plt.title("Decision Tree ROC Curve (Murtuza Shareef) - Excluding Class 3")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

#  Confusion Matrix
cm_dt = confusion_matrix(pred_dt_pd['label'], pred_dt_pd['prediction'], labels=[0,1,2])
plt.figure(figsize=(6,5))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Oranges', xticklabels=[0,1,2], yticklabels=[0,1,2])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree Confusion Matrix (Murtuza Shareef) - Excluding Class 3")
plt.show()
```





Task 5 – Results Visualization or Printing (5 Marks)

Random Forest (Shiva Reddy)

The Random Forest model results were picturized in order to easily explain the accuracy of prediction. A confusion matrix was provided to indicate the mis-classified transactions between classes. ROC curves were also plotted to evaluate the sensitivity-specificity trade-off of Classes 0.0, 1.0, and 2.0. Probability distributions of predictions were also printed out to note model confidence. These visualizations gave a clear idea about the behavior of the model in case of stronger results on the majority class and weaker results on the minority class.

KMeans Cluster (Mohammed Shoeb)

Since KMeans is an unsupervised model, visualization of the results consisted of cluster distribution plots. The print-out of number of transactions per cluster showed that Cluster 0 had most transactions, the others were rare or aberrant patterns (Clusters 1 and 2). These visualizations were useful in determining the possible outlier transactions and judging the compactness of clusters. Scatter plots plotting numerical variables (Amount, Year, Month, Day) against cluster updated were also created to hill up in interpreting the patterns of clustering.

LR Classifier (Rahul Reddy)

Predicted Logistic Regressions were plotted in confusion matrices and ROCs of Classes 0.0, 1.0, and 2.0. The model confidence has been examined by printing probability histograms of predicted classes. These visualizations emphasized that the model was effective in correctly classifying the majority label as well as where misclassification took place, most especially on the minority class. They also gave a definite indication regarding model performance in evaluation and reporting.

Decision Tree Classifier (Murtuza Shareef)

The results of Decision Tree were tabulated and plotted using confusion matrices, class-wise probability distributions and ROC curves of selected classes. Columns that led prediction at nodes in the tree were analyzed in terms of their interpretability, and features that were largely influential to prediction were identified. As visualizations have shown, Decision Tree model was more successful across the board, with the ability to pick the patterns accurately, both in majority and minority, and generating an interpretable result, which can guide further work.

Conclusion

A complete PySpark pipeline was executed in this coursework to cover data loading, preprocessing, data exploration, feature preparation, multi-class classification in the RandomForest classifier, unsupervised grouping in the KMeans classifier, hyperparameter tuning using TrainValidationSplit, and evaluation with complete metrics and visualisations. The tuned RandomForest model scored 72.22% on a held-out test sample and also gave per-class statistics: high accuracy on the dominant class and low accuracy on the minority classes. Clustering gave an opportunity of exploratory segmentation that can be used in a feature engineering aspect (Almarshad et al., 2023). The project serves as a good illustration of what it takes to bring PySpark to bear on large tabular data: the secrets of thoughtful preprocessing, the kind of trade-offs to make between tuning and runtime, the power of putting together supervised and unsupervised procedures to provide a global vision.

Task 6 - LSEP Considerations (5 marks)

In designing and enacting this project, considering the Legal, Social, Ethical, and Professional (LSEP) factors should be borne in mind to ascertain compliance obligations, accountability and a sense of equity regarding technical and non-technical requirements.

Legal Considerations: Data protection laws and laws governing intellectual property are observed to the latter in the development of the project. Datasets that may be used are sourced publicly, or through license repositories, and no copyright is infringed. Additionally, all the data structures, algorithms and reports developed will be original, and all external sources intended to be used have appropriate referencing (and suchlike) as per the provisions of GDPR and the policies of academic integrity (Dauletbak et al., 2021).

Social Considerations: In the societal dimension, the system is aimed at benefiting the end-users who will become more efficient and transparent in terms of handling data. The implementation does not have discriminatory or bias practices entrenched within it. Inclusivity was also provided by the project team designing the models and documentation which could be interpreted and used by a wide range of user groups with the least overlap in the digital divides (Iqbal et al., 2024).

Ethical Considerations: Responsible ethics were also observed, as all the contributions were acknowledged and no plagiarism has taken place. The results of the project were done with diligence and integrity where there was no selectivity of the results and data manipulation (Tenali and Babu, 2023). Also, the parameters of fairness, accountability, and trust were provided in order to make sure that the solution would not be misused or distorted in real life context.

Professional Considerations: Presentation of effective cooperation within a team was practiced by Mohammed Shoeb Ahmed (2957472), Shiva Reddy Vonteddu (2964063), Murtuza Shareef Mohammed (2972802) and Rahul Reddy Male (2962184) and also by being professional. Equal contribution was made by each member in regards to research, implementation, and documentation as per the university guidelines. The professional standard used included Project management techniques including frequent meetings, agreed upon deadlines, and the division of work which approximates to professional practice as per real world.

Task 7 - Convert ipynb to HTML for Turnitin submission [5 marks]

```python
# install nbconvert
!pip3 install nbconvert
```

```
Requirement already satisfied: nbconvert in /usr/local/lib/python3.12/dist-packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (4.13.4)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert) (6.2.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (5.8.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (3.0.2)
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (3.1.3)
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (0.10.2)
Requirement already satisfied: nbformat>=5.7 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from nbconvert) (25.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (2.19.2)
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages (from nbconvert) (5.7.1)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.7->nbconvert) (4.3.8)
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.12/dist-packages (from nbclient>=0.5.0->nbconvert) (6.1.12)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (4.25.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert) (2.7)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert) (4.14.1)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.4.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.36.2)
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.27.0)
Requirement already satisfied: pyzmq>=13 in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.1)
Requirement already satisfied: python-dateutil>=2.1 in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: tornado>=4.1 in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.4.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.1->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
```

```python
# convert ipynb to html and submit this HTML file
!jupyter nbconvert --to html /content/drive/Group_49_CRWK_CN7030.ipynb
```

```
[NbConvertApp] WARNING | pattern '/content/drive/Group_49_CRWK_CN7030.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exclude_input=True --TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                  to output to the directory of each notebook. To recover
                                  previous default behaviour (outputting to the current
                                  working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slideshow)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```